UDP-Client例程

UDP-Client例程
- 例程简介
- 代码解析
  - 函数 `connect_wifi`
  - 函数 `start_udp_client`
  - 发送测试消息
  - 关闭套接字
  - 主程序入口

UDP协议
- 一、UDP基础认知
  - 1.1 什么是UDP？
  - 1.2 UDP vs TCP 核心差异
- 二、UDP协议结构（图解）
  - 2.1 数据报格式
  - 2.2 头部结构
- 三、UDP关键工作机制
  - 3.1 无连接通信
  - 3.2 校验和计算
- 四、UDP高级特性
  - 4.1 多路复用与分解
  - 4.2 应用层可靠性实现
- 五、UDP典型应用场景
  - 5.1 实时音视频传输
  - 5.2 游戏通信
  - 5.3 网络服务发现
  - 5.4 物联网通信

例程简介

本节我们介绍如何将K230作为UDP服务器使用

我们打开这一节的例程代码，修改代码中连接WIFI信息部分修改为自己家的WIFI或者手机热点

```
sta.connect("WIFI SSID", "WIFI PASSWORD")  # 连接到指定WiFi / Connect to specified WiFi
```

然后我们打开网络调试助手, 输入本机的IP地址（用ipconfig查看）



接着我们回到代码，修改这一部分的代码 （修改为上面填写的IP地址和端口号）

```
# 设置服务器参数 / Set server parameters
server_ip = '192.168.2.89'
server_port = 8080
```

点击左下角的运行按钮，控制台会输出向服务器发送的信息

```
发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 1

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 2

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 3

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 4

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 5

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 6

发送字节数 / Bytes sent: 29
```

```
发送消息 / Sending: K230 UDP client send test 7

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 8

发送字节数 / Bytes sent: 29
发送消息 / Sending: K230 UDP client send test 9

发送字节数 / Bytes sent: 29
客户端已结束 / Client ended.
```

同时，我们会到网络调试助手，会发现这边也接收到了K230发来的信息

```
[2025-02-21 10:37:01.615]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 0


[2025-02-21 10:37:01.679]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 1


[2025-02-21 10:37:01.862]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 2


[2025-02-21 10:37:02.188]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 3


[2025-02-21 10:37:02.378]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 4


[2025-02-21 10:37:02.466]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 5


[2025-02-21 10:37:02.779]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 6


[2025-02-21 10:37:02.869]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 7


[2025-02-21 10:37:03.096]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 8


[2025-02-21 10:37:03.272]# RECV ASCII/30 from 192.168.1.253 :49153 <<<
K230 UDP client send test 9
```

代码解析

完整注释和代码请参考文件【源码汇总 / 11.Network / 04.udp / udp_client.py】

```
import socket
import os
import time
import network
```

在代码的开头，导入了必要的模块：

- `socket`：提供了网络通信的功能，支持TCP和UDP协议。
- `os`：提供与操作系统交互的功能，例如退出程序。
- `time`：提供时间相关的功能，例如延时。
- `network`：提供网络相关的功能，这里用于WiFi连接。

函数 `connect_wifi`

```
def connect_wifi(ssid="[WIFI SSID]", password="[WIFI PASSWORD]"):
    """
    连接WiFi并返回IP地址
    Connect to WiFi and return IP address

    参数 / Parameters:
    ssid: WiFi名称 / WiFi name
    password: WiFi密码 / WiFi password

    返回 / Returns:
    str: IP地址 / IP address
    """
```

该函数用于连接到指定的WiFi网络，并返回连接成功后的IP地址。它的参数包括WiFi的名称（SSID）和密码。

```
    wifi_station = network.WLAN(0)  # 创建WiFi站点对象 / Create WiFi station object
    wifi_station.connect(ssid, password)  # 连接到指定WiFi / Connect to specified
WiFi
```

这里创建了一个WiFi站点对象，并使用提供的SSID和密码连接到WiFi网络。

```
    # 等待直到获取到IP地址 / Wait until IP address is obtained
    while wifi_station.ifconfig()[0] == '0.0.0.0':
        os.exitpoint()
```

通过`ifconfig()`方法检查IP地址是否已获取。若未获取（即IP地址为`0.0.0.0`），则进入循环，不断检查，直到获取到有效的IP地址。此处的`os.exitpoint()`可能是一个错误，通常应该是`os._exit(1)`或`raise SystemExit`。

```
    return wifi_station.ifconfig()[0]  # 返回IP地址 / Return IP address
```

一旦获取到有效的IP地址，便将其返回。

函数 `start_udp_client`

```
def start_udp_client():
    """
    启动UDP客户端，发送测试消息
    Start UDP client and send test messages
    """
```

该函数负责启动UDP客户端并发送测试消息。

```
    # 连接WiFi网络 / Connect to WiFi network
    connect_wifi()
```

调用`connect_wifi`函数连接到WiFi。

```
    # 设置服务器参数 / Set server parameters
    server_ip = '192.168.2.89'
    server_port = 8080
```

定义要连接的UDP服务器的IP地址和端口。

```
    # 获取服务器地址信息 / Get server address information
    address_info = socket.getaddrinfo(server_ip, server_port)
    print("地址信息 / Address info:", address_info)

    server_address = address_info[0][-1]
    print("连接地址 / Connect address:", server_address)
```

使用`socket.getaddrinfo()`获取服务器的地址信息，并提取出实际的连接地址（元组的最后一个元素）。

```
    # 创建UDP套接字 / Create UDP socket
    udp_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

创建一个UDP套接字。

发送测试消息

```
    # 发送测试消息 / Send test messages
    try:
        for msg_count in range(10):
            # 构建消息 / Build message
            message = f"K230 UDP client send test {msg_count}\r\n"
            print("发送消息 / Sending:", message)

            # 发送消息并获取发送字节数 / Send message and get bytes sent
            bytes_sent = udp_socket.sendto(message.encode(), server_address)
            print("发送字节数 / Bytes sent:", bytes_sent)

            # 等待200ms / Wait 200ms
            time.sleep(0.2)
```

在`try`块中，一个循环发送10条测试消息：

- 消息内容采用格式化字符串，包含消息编号。
- 使用`sendto`方法发送消息，并打印发送的字节数。
- 每发送一条消息后，程序会等待200毫秒。

关闭套接字

```
    finally:
        # 关闭套接字 / Close socket
        udp_socket.close()
        print("客户端已结束 / Client ended.")
```

无论发送消息的过程是否成功，都会在`finally`块中关闭UDP套接字，并打印结束信息。

主程序入口

```
# 启动客户端 / Start client
if __name__ == '__main__':
    start_udp_client()
```

如果该脚本是主程序运行，则调用`start_udp_client()`函数启动UDP客户端。

UDP协议

一、UDP基础认知

1.1 什么是UDP？

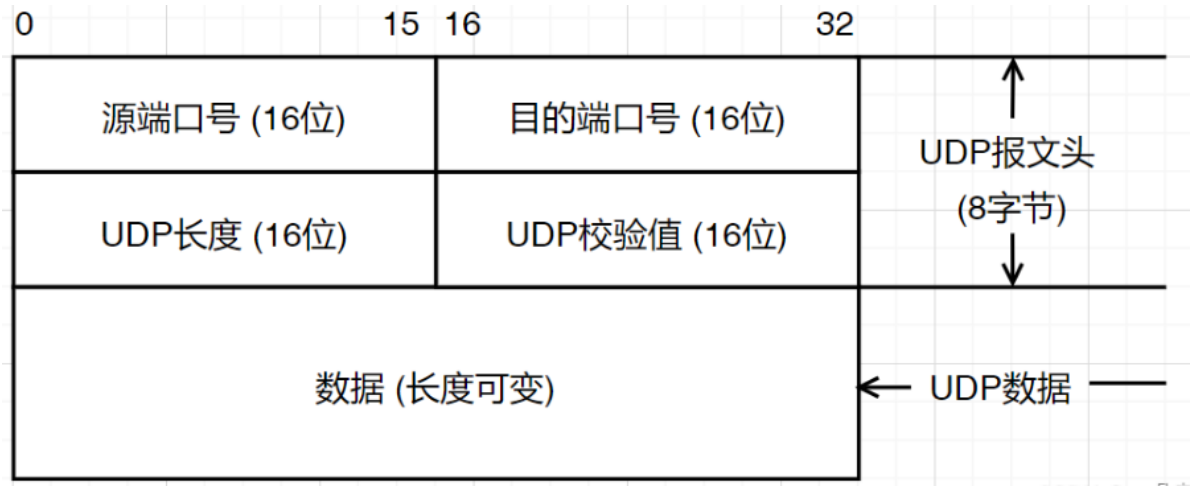
- **定位**：传输层协议（OSI第4层）
- **特点**：无连接、尽最大努力交付、轻量级头部、低延迟
- **类比**：像邮局的"普通明信片服务"（不保证必达，但投递速度快）

1.2 UDP vs TCP 核心差异

特性	UDP	TCP
连接建立	无握手	三次握手
数据传输保证	不保证顺序和到达	完全可靠（仅从本层来看，逻辑上可靠）
头部开销	8字节	20-60字节
传输控制	无流控/拥塞控制	精细控制
适用场景	实时通信	可靠传输

二、UDP协议结构（图解）

2.1 数据报格式



2.2 头部结构

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|          Source Port          |       Destination Port        |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|            Length             |           Checksum            |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                             Data                              |
+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
```

- **Source Port**（16位）：可选字段，0表示无源端口
- **Destination Port**（16位）：必填目标端口
- **Length**（16位）：头部+数据的字节总数（最小8字节）
- **Checksum**（16位）：校验和（IPv4可选，IPv6强制）

三、UDP关键工作机制

3.1 无连接通信

- 单播：一对一传输
- 组播：一对多传输（IP组播地址范围：224.0.0.0~239.255.255.255）
- 广播：一对所有传输（受限广播地址255.255.255.255）

3.2 校验和计算

- 覆盖范围：伪头部+UDP头部+数据
- 伪头部结构：

```
| 源IP地址（32位）| 目的IP地址（32位）| 协议（8位）| UDP长度（16位）|
```

四、UDP高级特性

4.1 多路复用与分解

- **端口号**：通过端口号区分不同应用（0~65535）

4.2 应用层可靠性实现

- **自定义重传**：如QUIC协议的丢包检测
- **前向纠错**：添加冗余数据应对丢包
- **序号机制**：在数据载荷中添加自定义序列号

五、UDP典型应用场景

5.1 实时音视频传输

- **Zoom/Teams**：允许少量丢包换取低延迟
- **流媒体协议**：RTP over UDP

5.2 游戏通信

- **MOBA类游戏**：高频位置更新（每秒20-30次）
- **FPS射击游戏**：快速传递操作指令

5.3 网络服务发现

- **DHCP**：动态获取IP地址
- **mDNS**：局域网设备发现（如打印机）

5.4 物联网通信

- **CoAP**：受限设备的轻量级协议
- **MQTT-SN**：传感器网络协议